

START WORDS

単語力は英語学習の出発点

主な機能一覧

フラッシュカード学習

カードをめくる直感的な
暗記学習モード

穴埋め記述モード

英単語を実際に入力して
「書いて覚える」定着化

サイトマップ

画面構成を一覧で把握アプリ全
体の流れを可視化

BB-8 トグル

学習をアニメーションで
楽しく演出するUI

単語帳 CRUD

単語の登録・単語帳の一覧表示

セッション認証

ログイン・会員登録で
ユーザー別に学習管理

私たちが力を入れたこと



AI駆動開発

AIをテックリードとして
位置づけ、以下に活用

- ・設計
- ・レビュー



アジャイル実践

1日1MVPのスプリントで
毎日「動くもの」をリリース



チーム運営

- ・シェアードリーダーシップ
- ・セマンティックバージョニング
- ・Gitの使用

私たちが目指したのは、「動くアプリ」ではなく「実務に近い開発経験」

AI駆動開発とは

AIをツールとして使うのではなく、開発チームの一員として組み込む開発手法

	一般的なAI駆動開発
コード生成	AIがコードを自動生成・補完
レビュー	AIがバグ・改善点を指摘
設計	AIが構成案・アーキテクチャを提案
責任所在	人間が最終確認・判断を行う

本チームの解釈: AIをテックリードに

テックリードとは

チームの技術的な方向性を決め、
メンバーの開発をサポートするエンジニア

- 1 単なるコーダーではなく「技術の責任者」
- 2 マネージャーではなく「現場で動くリーダー」
- 3 設計・レビュー・判断を担う役割

AIに担わせた役割

- ✓ 要件定義・役割分担の整理
- ✓ Mermaid図・ドキュメント生成
- ✓ コードレビュー・改善提案

AIに担わせなかった役割

- × 実装コードの生成
- × 最終的な技術判断
- × チーム間のコミュニケーション

上流工程でのAI活用

要件定義

- 目的・機能リストの整理
- Claudeとの対話で仕様を深掘り
- 欠員想定のスケジュール考案

設計

- Mermaidで画面遷移図生成
- Geminiデモの完成イメージ作成
- 複数案の比較・最適案の選択

上流工程でのAI活用

• 目的・機

• Claude

• 欠員想定

A	B	C	D
スケジュール			
Day	Ver	MVP内容予定	MVP内容実際
2	v0.1.0	単語登録・基本表示, 2名	単語登録・基本表示実装、ヘッダー・フッターも実装（仮）
3	v0.2.0	BB-8トグル操作・カードめくり, 2名	穴埋め記述モード実装・判定ロジック
4	v0.3.0	穴埋め記述モード・判定ロジック, 2名	ログインページ作成・トグルボタン実装
5	v0.4.0	正答率集計・苦手優先出題, 3名	ログイン・新規登録ページPHP実装とWord.phpのjQuery導入
6	v0.5.0	ログイン認証・マイページ統合, 3名	マイページ作成、セッション・クッキー管理実装
7	v0.6.0	学習グラフ・公開共有・ バッファ , 3名	UI調整など各リファクタリング、バグ洗い出し
8	v1.0.0	セキュリティ強化・ドキュメント作成, 3名	UI最終調整、本番環境構築、発表資料まとめ

+

≡

開発ルール

1 編集管理

変更履歴

役割担当

スケジュール

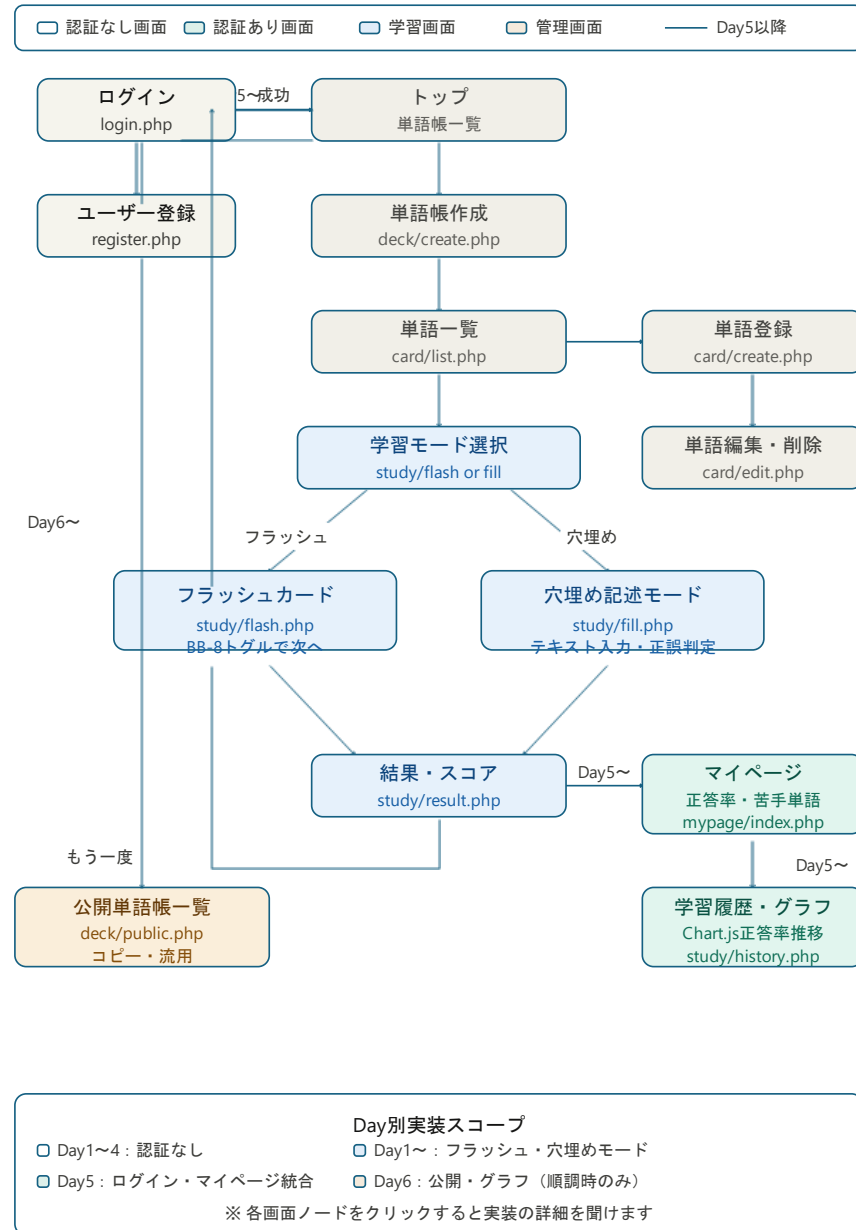
<

>

<

上流

- 目的
- Class
- 欠負



設計

- Mermaidで画面遷移図生成
- Geminiデモの完成イメージ作成
- 複数案の比較・最適案の選択

上流工程でのAI活用

1 / 5

ヒント：りんご

The is a fruit.

答え合わせ

BB-8を動かして次のカードへ



設計

Mermaidで画面遷移図生成

Geminiデモの完成イメージ作成

複数案の比較・最適案の選択

差別化案 6選

学習アルゴリズム系

案① **忘却曲線リマインダー** エビングハウスの忘却曲線をベースに「次に復習すべき日」を自動計算して表示。1日後・3日後・7日後・21日後のタイミングで復習を促す。「いつ復習すればいいか」を考えなくていい設計。

案② **苦手優先出題エンジン** 正解・不正解の履歴をもとにスコアを算出し、正答率の低いカードを自動的に優先表示。毎回ランダムではなく「弱点を集中攻略」できる。

ソーシャル・共有系

案③ **単語帳マーケットプレイス** 作成した単語帳を公開・共有できる。他ユーザーの単語帳をそのまま使ったり、コピーして自分用にカスタマイズできる。「一から作る手間ゼロ」で始められる。

案④ **対戦・タイムアタックモード** 同じ単語帳で友達と正答率や回答速度を競えるモード。学習を「ゲーム化」することで継続モチベーションを維持。

可視化・フィードバック系

案⑤ **学習ダッシュボード** 連続学習日数（ストリーク）・総学習枚数・カテゴリ別正答率をグラフで可視化。Duolingoのような「続けたくなる」仕掛け。

入力・体験系

案⑥ **穴埋め記述モード** フラッシュカードの「めくる」形式に加え、答えをテキスト入力して正誤判定する記述モードを搭載。「見て覚える」だけでなく「書いて覚える」を両立。

選ぶ目安

7日間・初～中級チームで実装しやすい順：

やりやすい ①忘却曲線 ②苦手優先 ⑤ダッシュボード

普通 ⑥穴埋め記述 ③マーケット

やや重い ④対戦モード

複数組み合わせも OK です。どれが刺さりますか？

AIコーチング: Claudeとの対話

プロンプト全文は共有フォルダをご参照ください — ここでは**工夫のポイント**のみ説明します

1

役割を明示する

「あなたはこのプロジェクトのテックリード（技術リーダー）です。」と役割を最初に定義。回答の質と一貫性を向上させた。

2

制約を与える

「コードを代わりに書かない」「1問ずつ質問する」など。役割と行動範囲を明示することで、AIの暴走を防ぎ、意図通りの支援に。

3

出力形式を指定する

フェーズ1～3という段階構造を定義し、「何をいつ・どの順序で出力するか」を明示。AIの返答がブレず、一貫したレビュー体験を実現した。

4

反復対話で深掘りする

1回のプロンプトで完結させず、対話を重ねることで情報を段階的に引き出す設計にした。

アジャイル開発の実践

アジャイル開発

短いサイクルで開発・検証を繰り返す
開発手法。
変化に素早く対応できることが特徴

スプリント

1～4週間の短い開発単位。
本チームでは「1日=1スプリント」として運用

スクラム

アジャイルの代表的なフレームワーク。
チームで役割を決め、短い周期で開発を
繰り返す手法

MVP

Minimum Viable Product。
(ミニマム・バイアブル・プロダクト)
まず動くものを作り、改善を重ねる手法

1日のスクラムの流れ

09:15~09:25

朝のスタンドアップ



- ・ 前日の振り返り
- ・ 追加機能の提案
- ・ 進捗と課題の共有

09:25~14:40

設計・実装フェーズ



各自のタスクを
集中実装

14:50~15:15

成果確認



- ・ 実装内容の共有
- ・ 日誌の記入

15:15~15:30

フィードバック



AIによる
コードレビュー

15:30~15:40

デイリーリリース



- ・ MVPの動作確認
- ・ タグ付けリリース

スプリントの改善

初日の問題点

- × 朝のスタンドアップの時間が定まっていなかった
- × AIにコードレビューのみ実施させる



翌日以降の改善

- ✓ 朝のスタンドアップの時間を5～10分間と設定
- ✓ AIコーチングプロンプトをテンプレート化

シェアードリーダーシップ

リーダー1人がすべての責任を背負うのではなく、
チームの構成員それぞれが場面に応じて
リーダーシップを発揮し、権限も含めて分担する考え方



チームでは「1人が全責任を負う体制」ではなく、
得意分野に応じて権限ごと分担する体制

セマンティックバージョンング

メジャー: (例: 1.0.0 → 2.0.0) 既存の機能が使えなくなるような大きな変更

マイナー: (例: 1.0.0 → 1.1.0) 既存機能はそのままで、新しい機能を追加

パッチ: (例: 1.0.0 → 1.0.1) 問題修正のみを行った場合

変更の重要度を明確にし、チームの共通言語とします。

セマンティックバージョンニング

本チームでのバージョン番号のルール

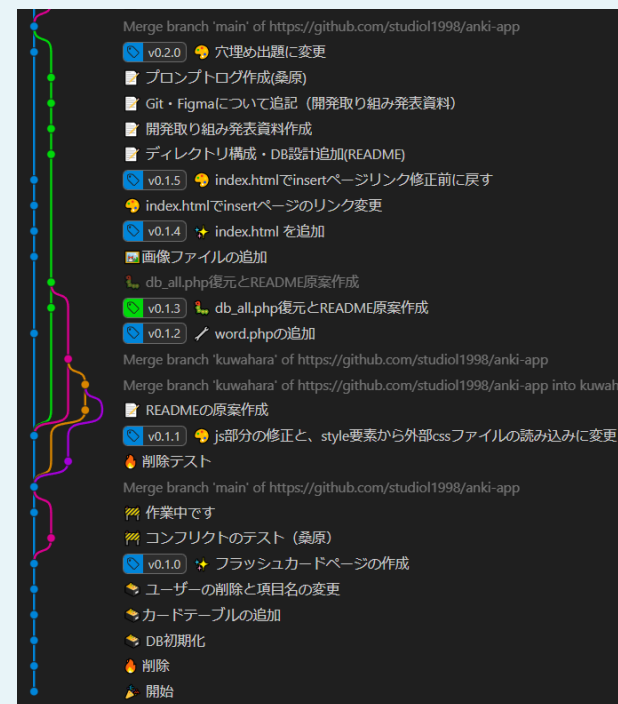
MAJOR . **MINOR** . **PATCH**

MAJOR 最終日リリース

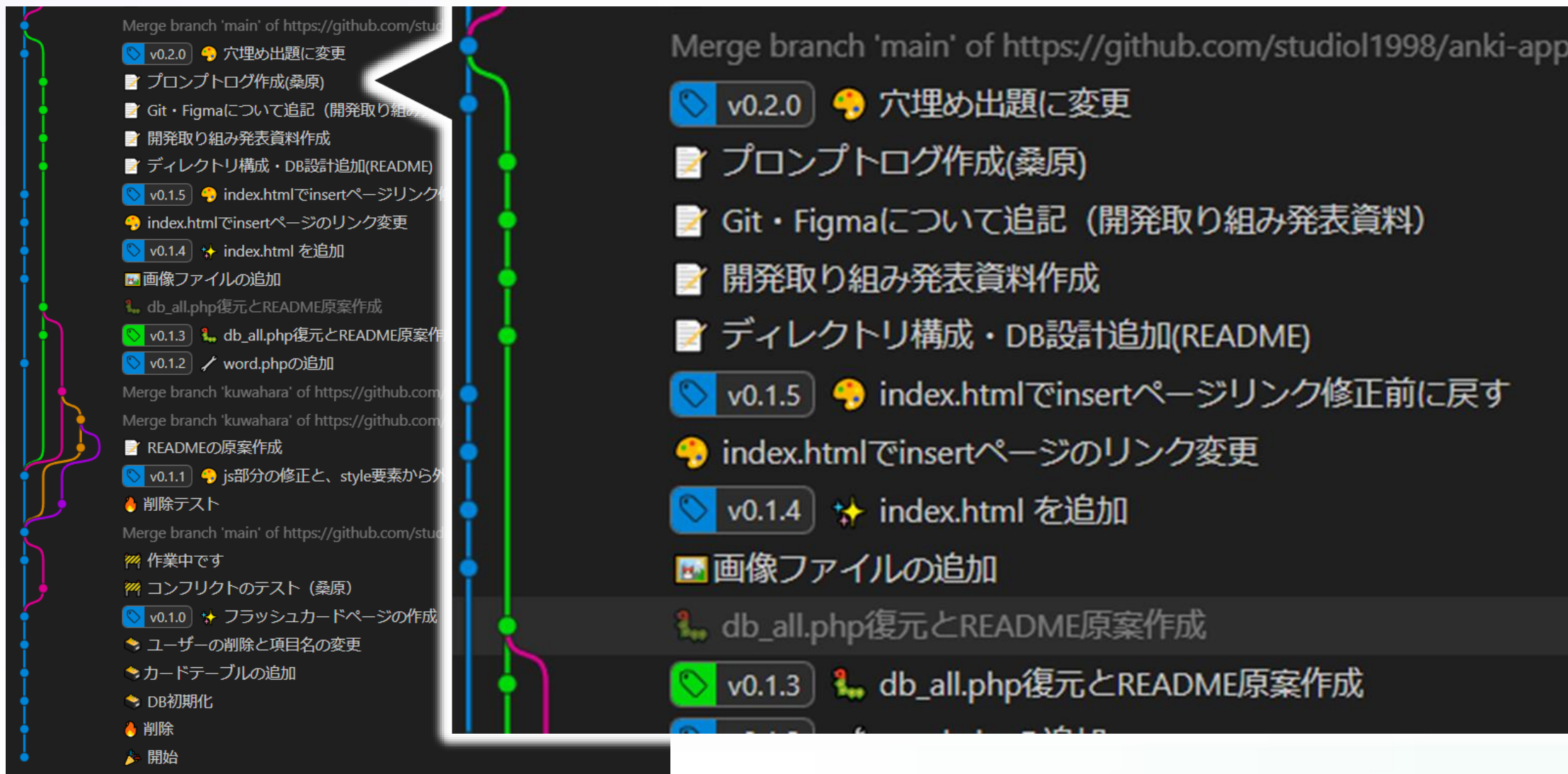
MINOR 1日の目標達成でインクリメント

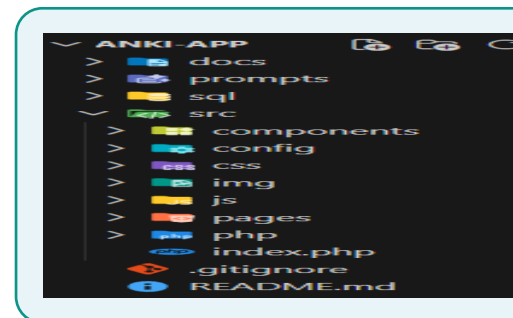
PATCH UI/UXの変更でインクリメント

Gitタグログ



Gitタグログ



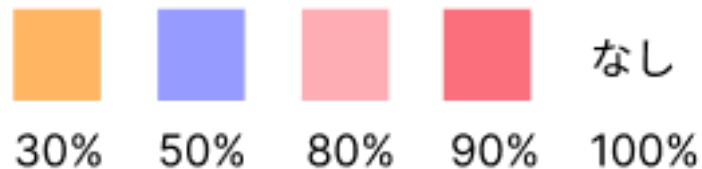


ツール・ルール選定

Figma

- ・フローチャート作成
- ・画面遷移図作成

【進捗管理タグ】



【凡例】

- ・ 30%：設計が完了し、ファイルを作った
- ・ 50%：プログラムの基礎構築が完了した
- ・ 80%：おおまかに完成し、微調整に入った
- ・ 90%：ほぼ完成し、最終調整中。テスト前
- ・ 100%：完成！
基本的には変更等はしない
★classの衝突等による変更等は
この限りではない。

スプレッドシート

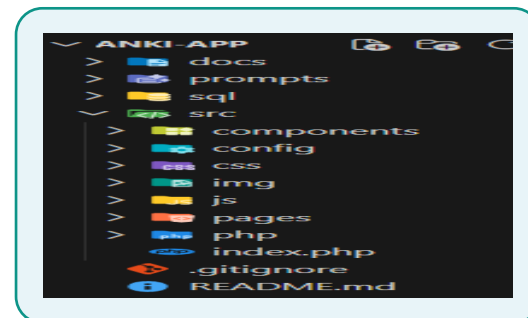
- ・開発ルール
- ・変更履歴
- ・タスク管理
- ・参考サイトリンクなど

ファイルの変更履歴を記録・管理する
バージョン管理ツール

「いつ・誰が・何を変えたか」を追跡できる
過去の状態にいつでも戻すことができる

レクトリ構成

を想定した
レクトリ構成を

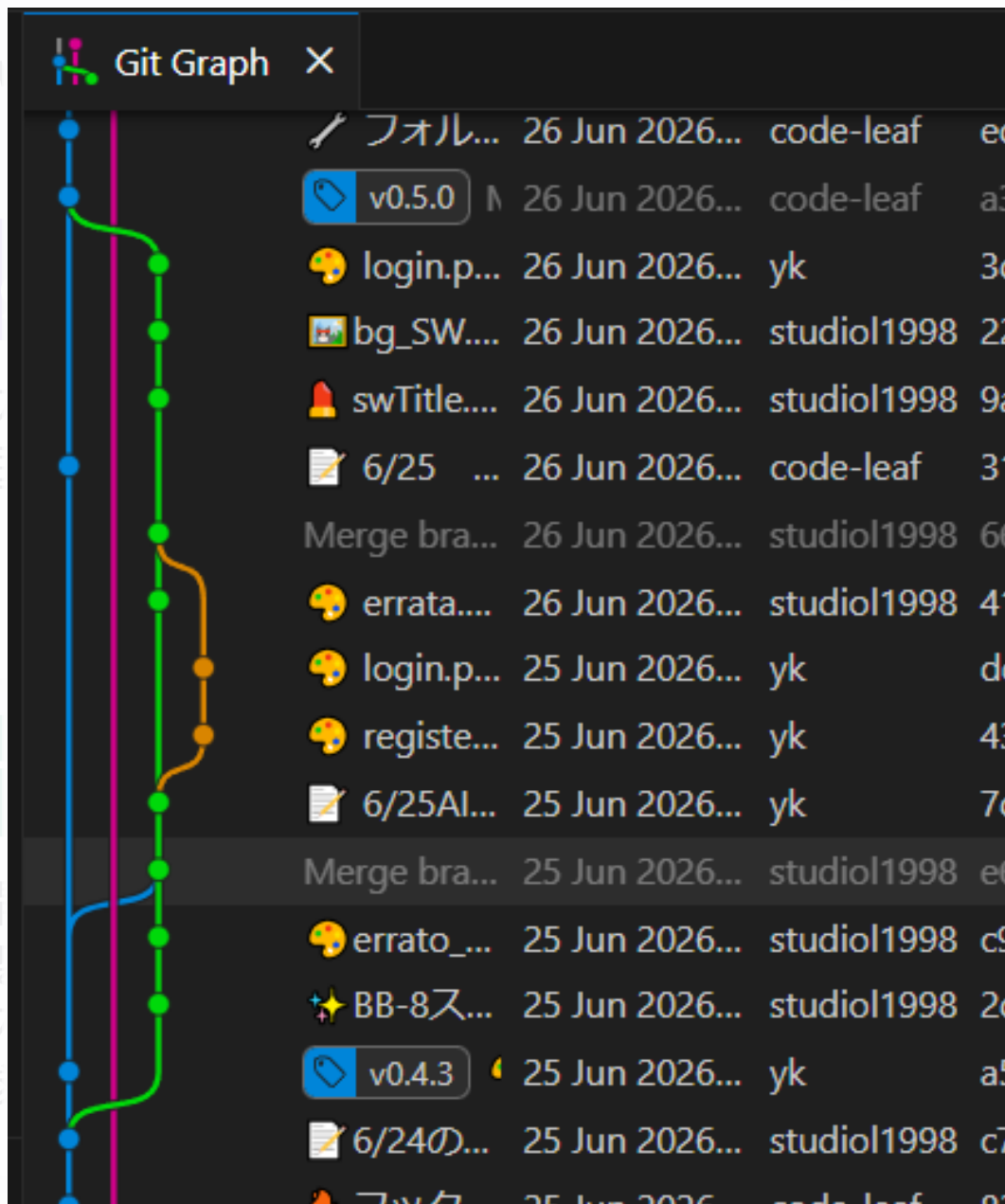


Fi

・フ
・画

ス

・開
・変
・タ
・参

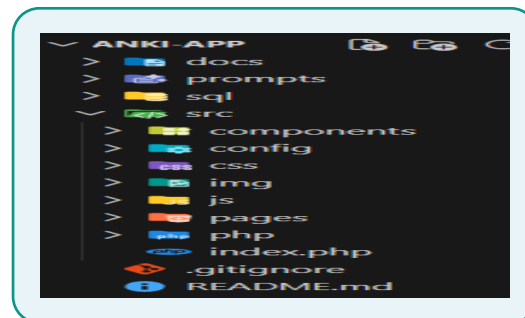


Git

- ファイルの変更履歴を記録・管理するバージョン管理ツール
- 「いつ・誰が・何を変えたか」を追跡できる
- 過去の状態にいつでも戻すことができる

ディレクトリ構成

実務を想定した
ディレクトリ構成を
採用



ツール・ルール

Figma

- ・フローチャート作成
- ・画面遷移図作成

スプレッドシート

- ・開発ルール
- ・変更履歴
- ・タスク管理
- ・参考サイトリンクなど

開発ルール	ジャンル	内容	備考
	コーディング	作業前	編集中管理シートを確認後、記載がなければ記入
	コーディング	作業前	編集中管理シートを確認後、記載がなければ担当者と競合しないように打ち合わせ
	コーディング	作業中	Figmaの画面遷移図の進捗タグの管理（適宜）
	コーディング	作業後	編集中管理シートの該当行のステータスを完了にし、変更履歴シートを更新
	AI活用	禁則事項（備考参照）	AIにコードを生成させない
	AI活用	コード完成後（備考参照）	AIにコードレビューさせ、フィードバックを受ける
	AI活用	1日の終わり（備考参照）	残タスクの洗い出し・優先順位の提案させる
	AI活用	許可項目（備考参照）	各種ドキュメント・コメントの自動生成・命名案の提案
	命名規則	ファイル名・フォルダ名	スネークケース（例: db_all.php）
	命名規則	id名	キャメルケース（例: cardFront）
	命名規則	クラス名	ケバブケース（例: fade-in）
	命名規則	JS_変数	キャメルケース（例: cardFront）
	命名規則	PHP_変数	「\$」プレフィックス付きキャメルケース（例: \$userAnswer）
	命名規則	SQL_変数	スネークケース（例: users_id）
	命名規則	定数	アッパースネークケース（例: ）
	GitMoji	🔥	削除関連
	GitMoji	📦	DB関連
	GitMoji	🧠	UI関連（🧠の使用は控える ※ 本来は🧑🏻）
	GitMoji	📄	ドキュメント関連
	GitMoji	🖼️	画像
	GitMoji	👤	.gitignore（プライベート設定: 隠しファイル・フォルダ設定）
	GitMoji	🔗	不具合修正

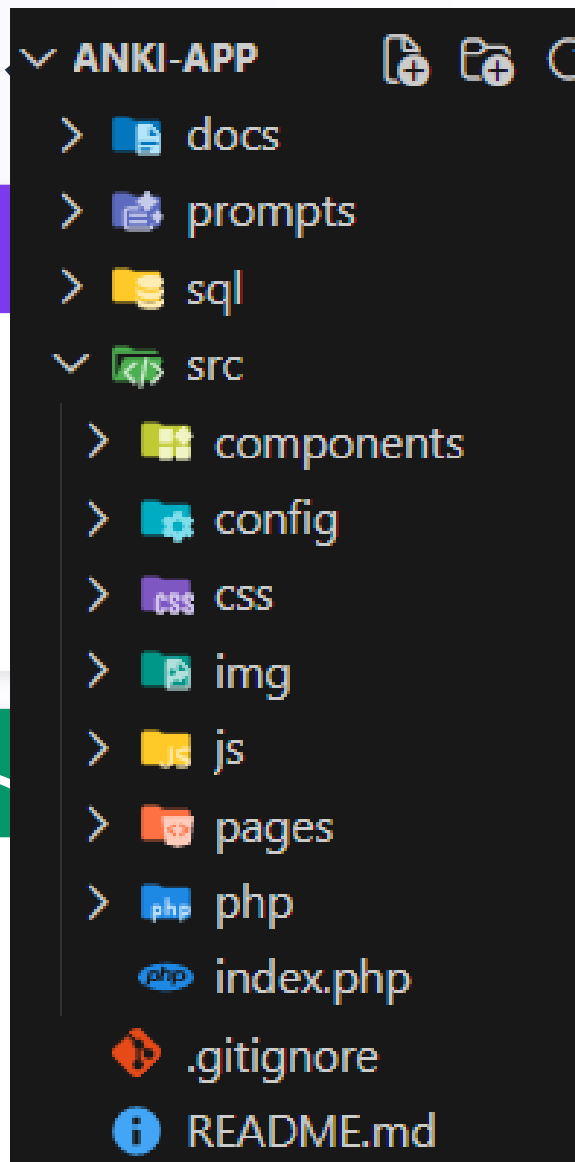
ツール・ルール

Figma

- ・フローチャート作成
- ・画面遷移図作成

スプレッドシート

- ・開発ルール
- ・変更履歴
- ・タスク管理
- ・参考サイトリンクなど

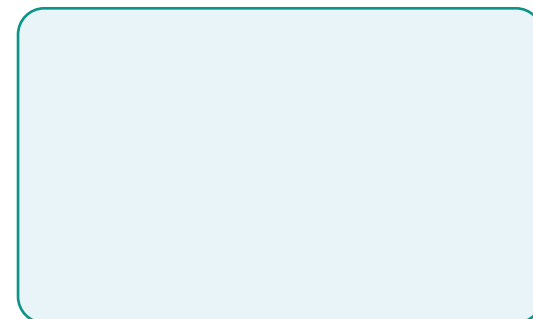


Git

- ・ファイルの変更履歴を記録・管理するバージョン管理ツール
- ・「いつ・誰が・何を変えたか」を追跡できる
- ・過去の状態にいつでも戻すことができる

ディレクトリ構成

実務を想定した
ディレクトリ構成を
採用



DRY原則・関心の分離

DRY 原則

Don't Repeat Yourself

同じコードを繰り返し書かない原則。

【本プロジェクトでの適用例】

- db_all.php でPDOオブジェクトをrequire
- 共通フッター・モーダル・トグルを分離
- カードフリップ処理の共通化

共通処理は関数・includeにまとめる

関心の分離

Separation of Concerns

役割ごとにファイルを分ける設計原則。

【本プロジェクトのディレクトリ構成抜粋】

/src/pages	画面表示ファイル
/src/php	データベース処理ロジック
/src/js	UI制御等
/src/css	スタイル定義

振り返り:もっと早くやればよかったこと

1

AIコーチングの早期導入

序盤からテックリードプロンプトを活用すれば、個人差のあるコード品質を早い段階で揃えられたはず

2

仕様書ルールの一貫性

ドキュメントのフォーマットを継続運用できる設計しておけば、形骸化せず長期的に運用できた

3

テンプレート運用の早期化

プロンプトテンプレート・コードテンプレートを初日に用意し開発すれば、品質のばらつきを防げた

4

DRY原則の早期徹底

後半にリファクタリングが発生した。最初から関心の分離を意識した設計をすべきだった

まとめ

1

AI駆動開発

AIをテックリードに位置づけ
上流～実装後まで一貫して活用

2

アジャイル実践

1日1MVPで毎日リリース
スプリントで継続的に改善

3

実務水準の運営

Semantic Versioning・
シェアードリーダーシップを導入

私たちが目指したのは、「動くアプリ」ではなく「実務に近い開発経験」

他のチームが「何をつくるか」に注力する中、本チームは「どうつくるか」にこだわりました。
AI駆動開発・アジャイル実践・実務水準の運営——この3つを軸に、実務の現場を
再現することに挑戦しました。